

// IBM Data Science Capstone

SpaceX Falcon9

Predictive Analysis of First Stage Landings:
A case study on reusability and aerospace efficiency.

github.com/bonetti1

linkedin.com/in/pedro-bonetti

PREPARED BY PEDRO BONETTI

// Context

Objective

This is the final project for the IBM Data Science Professional Certificate, covering all tasks — from API handling and Exploratory Data Analysis with SQL and Python, to result prediction with ML models.

The main goal is to build an ML model to predict landing success, cutting costs through rocket reusability and gaining a competitive edge. This allows for more accurate commercial proposals and reduces the risk of multi-million dollar losses by spotting unfavorable launch conditions ahead of time.

[README.md](#)

PREPARED BY PEDRO BONETTI

Data Collection: REST API

```
To make the requested JSON results more consistent, we will use the following static response object for this project:

static_json_url="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE0321EN-SkillsNetwork/datasets/API_call_spacex_api.json"

We should see that the request was successful with the 200 status response code

response=requests.get(static_json_url)

response.status_code

200

Now we decode the response content as a json using .json() and turn it into a Pandas dataframe using .json_normalize()
```

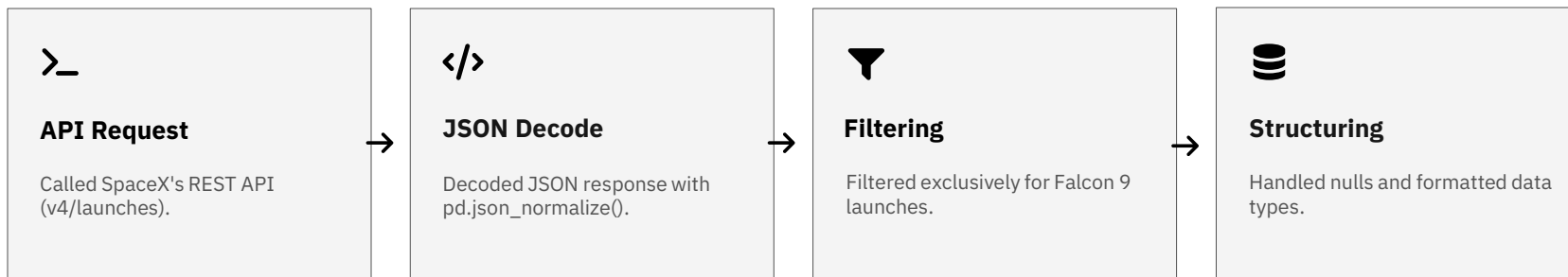
```
Task 3: Dealing with Missing Values

Calculate below the mean for the PayloadMass using the .mean(). Then use the mean and the .replace() function to

# Calculate the mean value of PayloadMass column
mean_payload_mass = data_falcon9['PayloadMass'].mean()
print(f"Mean PayloadMass: {mean_payload_mass:.2f} kg")

# Replace the np.nan values with the mean value
data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].replace(np.nan, mean_payload_mass)

# Verify no more missing values in PayloadMass
print(f"Missing values in PayloadMass: {data_falcon9['PayloadMass'].isnull().sum()}")
```



Data Collection: Web Scraping

```
TASK 1: Request the Falcon9 Launch Wiki page from its URL

First, let's perform an HTTP GET method to request the Falcon Launch HTML page, as an HTTP response.

# use requests.get() with static url and headers
response = requests.get(static_url, headers=headers)
print(response.status_code)

✓ 13s

200

Create a BeautifulSoup object from the HTML response

# Create BeautifulSoup object from response text
soup = BeautifulSoup(response.text, 'html.parser')

✓ 0s

Print the page title to verify if the BeautifulSoup object was created properly

# Print soup title to verify
print(soup.title)
```

```
TASK 3: Create a data frame by parsing the launch HTML tables

We will create an empty dictionary with keys from the extracted column names in the previous task. Later, this dictionary will be converted into a DataFrame.

launch_dict = {}

# Iterate over the HTML tables
for table in soup.find_all('table'):
    # Extract table headers
    headers = table.find('tr').find_all('th')
    launch_dict[headers[0].text] = []

    # Iterate over table rows
    for row in table.find_all('tr')[1:]:
        # Extract table row data
        row_data = row.find_all('td')
        launch_dict[headers[0].text].append(row_data)

# Print the launch_dict
print(launch_dict)

✓ 0s

Next, we just need to fill up the launch_dict with launch records extracted from table rows.

Usually, HTML tables in Wiki pages are likely to contain unexpected annotations and other types of noises, such as reference links. To simplify the parsing process, we have provided an incomplete code snippet below to help you to fill up the launch_dict. Please complete it.
```

```
df = pd.DataFrame({key:pd.Series(value) for key, value in launch_dict.items() })

df.to_csv('spacex_web_scraped.csv', index=False)
print('Saved spacex_web_scraped.csv')
df.head()
```

Flight No.	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Version	Booster	Booster lot
0	1	CCAFS Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	F9 v1.0	800001.18	
1	2	CCAFS Dragon	0	LEO	NASA	Success	F9 v1.0	800004.18	
2	3	CCAFS Dragon	525 kg	LEO	NASA	Success	F9 v1.0	800005.18	No att
3	4	CCAFS SpaceX CRS-1	4,700 kg	LEO	NASA	Success	F9 v1.0	800006.18	No att
4	5	CCAFS SpaceX CRS-2	4,877 kg	LEO	NASA	Success	F9 v1.0	800007.18	No att

We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this last section, following labs will be using a provided dataset to make each lab independent.



Data Treatment: **Wrangling**

TASK 2: Calculate the number and occurrence of each orbit

Use the method `.value_counts()` to determine the number and occurrence of each orbit in the column `Orbit`.

Note: Do not count `GTO`, as it is a transfer orbit and not itself geostationary.

```
# Apply value_counts on Orbit column
df['Orbit'].value_counts()
```

✓ 0.0s

Orbit

GTO 27

ISS 21

VLEO 14

PO 9

LEO 7

SSO 5

MEG 3

MEG 1

ES-L1 1

SO 1

TASK 4: Create a landing outcome label from Outcome column

Using the `Outcome`, create a list where the element is zero if the corresponding row in `Outcome` is in the set `bad_outcome`; otherwise, it's one. Then assign it to `landing_class`.

```
# landing_class = 0 if bad_outcome
# landing_class = 1 otherwise
bad_outcome = {'None None', 'False ASDS', 'False Ocean', 'None ASDS', 'False RTLS'}
landing_class = [0 if outcome in bad_outcome else 1 for outcome in df['Outcome']]
```

✓ 0.0s

This variable will represent the classification variable that represents the outcome of each launch. If the value is zero, the first stage did not land successfully; one

```
df['Class'] = landing_class
df[['Class']].head(5)
```

✓ 0.0s

Class



Missing Values

Identified missing data with `.isnull()`.



Imputation

Calculated the mean and filled `PayloadMass`.



Binary Mapping

Converted outcomes into a `Class` variable (1=Success, 0=Failure).



Encoding

Applied One-Hot Encoding to categorical variables.

Exploratory Analysis: SQL

```
import csv, sqlite3
import prettytable
prettytable.DEFAULT = 'DEFAULT'

con = sqlite3.connect("my_data1.db")
cur = con.cursor()
```

✓ 0.0s

```
!pip install -q pandas
```

✓ 1.3s

```
%sql sqlite:///my_data1.db
```

✓ 0.0s

```
import pandas as pd
```

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground p

```
%%sql
SELECT "Landing_Outcome", COUNT(*) AS Outcome_Count
FROM SPACEXTABLE
WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY "Landing_Outcome"
ORDER BY Outcome_Count DESC;
```

19] ✓ 0.0s

```
* sqlite:///my_data1.db
Done.
```

Landing_Outcome	Outcome_Count
No attempt	10

Launch Volume: CCAFS SLC-40 handles the highest total volume of operations.

Payload Insights: Queries identified total payload mass and isolated critical weight ranges.

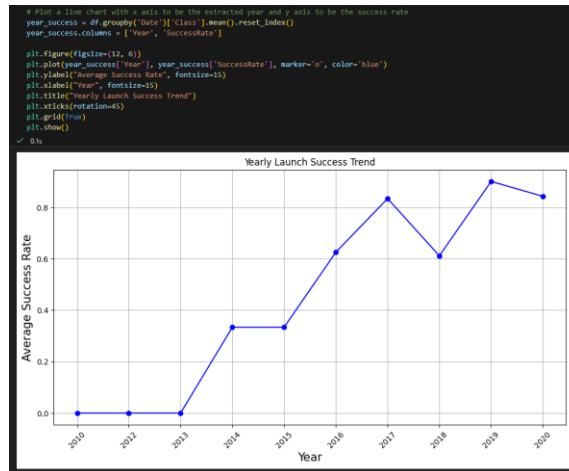
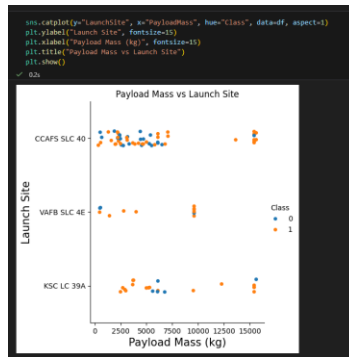
Historical Milestones: Pinpointed the exact first successful ground landing.

Exploratory Analysis: Visualization

Flight Number: Success rate improves as flight history grows (operational maturity).

Successful Orbits: ES-L1, GEO, HEO, and SSO orbits hold a 100% historical success rate.

Payload Mass: Heavy payloads are mostly operated from KSC LC-39A.



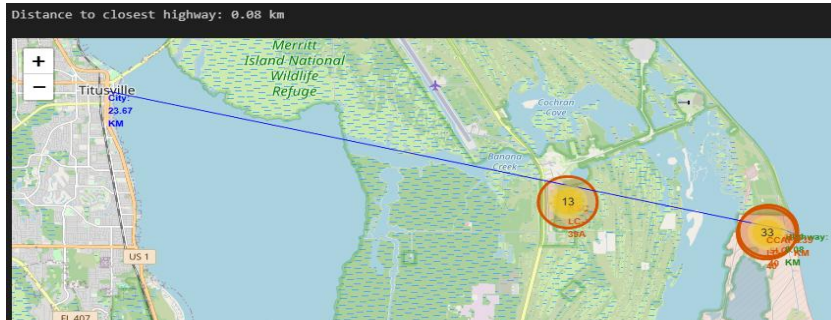
Geospatial Analysis: Folium Maps



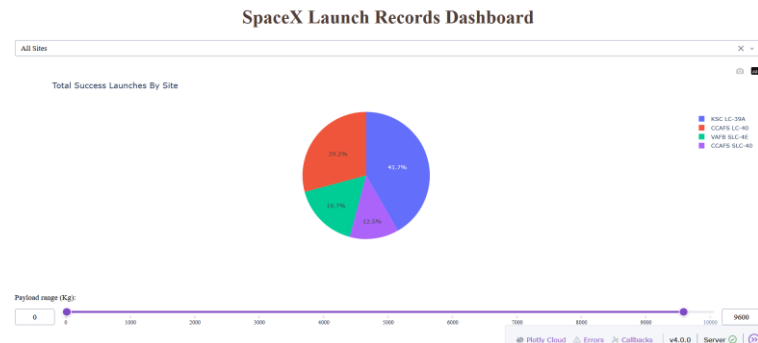
Coastal Positioning: Strategic location for safe ocean debris drop zones.

Logistics: Extremely close (< 1km) to railways and highways for booster transport.

Safety: Calculated distance from densely populated urban areas.



Interactive Dashboard: Plotly Dash



Efficiency: KSC LC-39A has the best success-to-launch ratio.

[spacex_dash.py](#)

Total Success Launches for site KSC LC-39A



Sweet Spot: Payloads between 0–5,000 kg show the highest concentration of safe landings.

Predictive Analysis: Classification

Logistic Regression

83.3%

SVM

83.3%

Decision Tree

83.3%

KNN

83.3%

Accuracy: All models tuned via GridSearchCV reached 83.3% precision on the test set.

Objectives

Perform exploratory Data Analysis and determine Training Labels

- create a column for the class
- Standardize the data
- Split into training data and test data

-Find best Hyperparameter for SVM, Classification Trees and Logistic Regression

- Find the method performs best using test data

TASK 3

Use the function `train_test_split` to split the data X and Y into training and test data. Set the parameter `test_size` to 0.2 as

```
X_train, X_test, Y_train, Y_test
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)
```

we can see we only have 16 test samples.

```
Y_test.shape # Should output (16,)
# 16,
```

TASK 4

Create a logistic regression object then create a `GridSearchCV` object `logreg_cv` with `cv = 10`. Fit the object to find the

```
parameters = {'C': [0.01, 0.1, 1, 10],
              'penalty': ['l1', 'l2'],
              'solver': ['lbfgs']}
```

```
# Create logistic Regression object
lr = LogisticRegression()

# Create GridSearchCV object with cv=10
```

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv`

```
parameters = {'criterion': ['gini', 'entropy'],
              'splitter': ['best', 'random'],
              'max_depth': [2**n for n in range(1,10)],
              'max_features': ['auto', 'log'],
              'min_samples_leaf': [15, 2, 4],
              'min_samples_split': [2, 5, 10]}

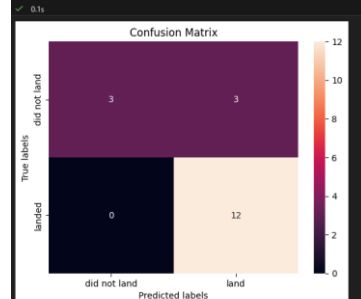
tree = DecisionTreeClassifier()
```

```
# Create GridSearchCV object with cv=10
tree_cv = GridSearchCV(tree, parameters, cv=10)

# Fit to find best parameters
tree_cv.fit(X_train, Y_train)
```

```
print("Used hyperparameters: (best parameters) ", tree_cv.best_params_)
print("accuracy -> ", tree_cv.best_score_)
```

```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```



Project Conclusions

Strategic Site: KSC LC-39A offers the highest landing success rate.

Learning Curve: Accuracy grows with flight number, validating SpaceX's iterative engineering approach.

High Predictability: The ML models predict outcomes with 83.3% accuracy.

Business Pick: The Decision Tree is recommended for its ease of explaining results to executives.